

Circuit-Based Pruning: From Interpretability Scores to Sparsity Budgets in Large Language Models

Uri Z. Kialy

Abstract

Unstructured pruning methods for large language models, such as Wanda/Wanda++, apply a uniform sparsity ratio across all layers, treating every layer as equally compressible. We argue that this assumption is fundamentally misguided: layers differ dramatically in their functional importance to the model’s computational circuits. We propose **Circuit-Guided Pruning**, a method that uses mechanistic interpretability—specifically, circuit discovery via Relevance Patching (RelP)—to derive a functional importance score for each layer, and then uses these scores to allocate non-uniform, layer-wise sparsity budgets. Within each layer, the weight selection uses Wanda’s efficient $|W| \cdot \|X\|$ criterion. On LLaMA-3.2-3B, our method matches Wanda at moderate sparsity (30%) and outperforms it where it matters most—at aggressive sparsity levels, reducing perplexity by 14.7% at 50% sparsity and by 39.4% at 70% sparsity, and improving average zero-shot accuracy by 1.1 percentage points at 50% and 2.6 points at 70% across seven benchmarks. Ablations confirm that the circuit-derived scores carry genuine signal: they outperform both a U-shaped heuristic and randomly shuffled layer scores, and remain effective even without manual layer-protection heuristics. This work bridges mechanistic interpretability and model compression, using circuit discovery as a sparsity prior for weight pruning. [ur code is publicly available at: https://github.com/UriKialy/models_backbone_via_circuit_discovery.git](https://github.com/UriKialy/models_backbone_via_circuit_discovery.git)

1 Introduction

The deployment of large language models (LLMs) is constrained by their enormous memory and compute requirements. Pruning—setting a fraction of model weights to zero—is among the most practical compression strategies, as it requires no re-training and can be applied post-hoc to pretrained

models (Hassibi et al., 1993; Han et al., 2015). Recent one-shot pruning methods such as SparseGPT (Frantar and Alistarh, 2023) and Wanda (Sun et al., 2024) have demonstrated that 50% or more of weights can be removed from billion-parameter models with tolerable perplexity degradation.

However, virtually all existing pruning methods share a critical simplifying assumption: they apply the *same sparsity ratio uniformly across all layers*. This assumption treats every transformer layer as equally compressible, ignoring the well-documented finding from mechanistic interpretability that layers serve qualitatively different computational roles (Elhage et al., 2021; Olsson et al., 2022; Wang et al., 2022). Early layers handle token embedding and positional encoding; middle layers implement abstract reasoning circuits; late layers project to the output vocabulary. There is no reason to expect these components to be equally robust to pruning.

Meanwhile, a parallel line of work in mechanistic interpretability has developed powerful tools for identifying the computational circuits within neural networks (Conmy et al., 2023; Syed et al., 2024; Jafari et al., 2025a). Edge Attribution Patching with Integrated Gradients (EAP-IG) (Hanna et al., 2024) can score every edge in the computational graph by its causal contribution to model behavior; these edge scores can be aggregated to obtain node-level (attention head, MLP block) importance to model behavior. More recently, Relevance Patching (RelP) (Jafari et al., 2025a) improves upon standard attribution patching by replacing noisy local gradients with propagation coefficients derived from Layer-wise Relevance Propagation (LRP) (Bach et al., 2015), yielding more faithful component attributions at the same computational cost. These tools have primarily been used for *understanding* models, but their outputs—quantitative importance scores over model components—are precisely the information that non-uniform pruning

needs.

In this paper, we bridge these two fields. We propose **Circuit-Guided Pruning**, a two-stage method that:

1. **Discovers functional importance** via circuit attribution. We run ReLP on a calibration corpus to score each attention head and MLP block by its causal contribution to language modeling. ReLP, which replaces the gradients in attribution patching with LRP-derived propagation coefficients, produces the layer-level importance scores used in our final method.
2. **Allocates non-uniform sparsity** using a softmax-temperature scheme that converts layer importance scores into per-layer sparsity budgets, protecting critical layers while aggressively pruning less important ones.
3. **Prunes locally via Wanda** within each layer, using the established $|W| \cdot \|X\|$ importance criterion to select which specific weights to remove given that layer’s budget.

A key technical challenge is adapting circuit discovery—which was designed for narrow, well-defined tasks like the Indirect Object Identification (IOI) circuit (Wang et al., 2022)—to work on *general-purpose language modeling*. Unlike IOI, where corrupted inputs are created by swapping named entities, there is no single “clean vs. corrupted” distribution for open-domain text. We address this by constructing corrupted inputs via token-level shuffling combined with Gaussian embedding noise, which disrupts the model’s next-token prediction on naturalistic text from the Pile (Gao et al., 2020) without requiring any task-specific labeled data.

Our experiments on LLaMA-3.2-3B (Grattafiori et al., 2024) demonstrate that circuit-guided pruning using ReLP-derived scores matches uniform Wanda at 30% sparsity and increasingly outperforms it at higher sparsity, with the crossover occurring around 35%. At 50% sparsity, our method achieves a perplexity of 16.78 versus Wanda’s 19.66 (−14.7%); at 70%, 1072.98 versus 1769.44 (−39.4%). Five ablation studies confirm that these gains are driven by the specific circuit-derived layer ranking rather than by artifacts of non-uniform allocation itself.

2 Related Work

Unstructured pruning for LLMs. Classical pruning dates back to Optimal Brain Damage (Hassibi et al., 1993) and Optimal Brain Surgeon (?), which use second-order information to identify removable weights. Scaling these ideas to LLMs, SparseGPT (Frantar and Alistarh, 2023) formulates pruning as a sparse regression problem solved layer-by-layer with approximate Hessian updates. Wanda (Sun et al., 2024) simplifies this to a training-free criterion that scores each weight by $|W_{ij}| \cdot \|X_j\|_2$, where $\|X_j\|_2$ is the L_2 norm of the j -th input feature’s activations computed on a small calibration set. Both methods apply the same sparsity ratio to every linear layer, an assumption we relax. Concurrent work on non-uniform sparsity allocation includes OWL (Yin et al., 2025), which uses activation outlier statistics to set per-layer ratios—a complementary signal to our circuit-based approach. Concurrent work Wanda++ (Yang et al., 2025) improves the within-layer pruning criterion by incorporating decoder-block-level regional gradients and regional optimization, achieving up to 32% perplexity reduction over Wanda at 2:4 sparsity. Their approach is orthogonal to ours: Wanda++ improves *which weights to prune within a layer*, while our method improves *how much to prune each layer*. The two could be combined by using Wanda++’s RGS criterion as the local pruning stage in our framework.

Mechanistic interpretability and circuit discovery. The circuits framework (Elhage et al., 2021; Olsson et al., 2022) decomposes neural networks into interpretable computational subgraphs. Activation patching (Vig et al., 2020; Meng et al., 2022) and its efficient approximation, Attribution Patching (Nanda and Bloom, 2023; Syed et al., 2024), identify causally important components by measuring how model outputs change when individual activations are corrupted. EAP-IG (Hanna et al., 2024) extends this with Integrated Gradients (Sundararajan et al., 2017) for more faithful attribution. More recently, Relevance Patching (ReLP) (Jafari et al., 2025a) replaces the local gradient term in attribution patching with propagation coefficients from Layer-wise Relevance Propagation (LRP) (Bach et al., 2015), using transformer-specific rules (Ali et al., 2022; Jafari et al., 2025b) to achieve more faithful attribution at the same computational cost as standard attribution patching. ReLP has been shown to more accurately approximate activation

patching than gradient-based methods, particularly for MLP outputs and the residual stream.

Interpretability for compression. The idea that interpretability can inform compression is intuitive but underexplored. Prior work has used attention entropy (Voita et al., 2019) or neuron activation patterns (Dalvi et al., 2020) to identify redundant heads or neurons for structured pruning. However, these approaches operate at the level of individual component statistics rather than *functional circuit importance*—i.e., how much a component contributes to the model’s actual computation, as opposed to how “active” it appears in isolation. Concurrent work by Hatefi et al. (2025) uses LRP-based attribution for pruning and circuit discovery in LLMs, but targets structured pruning of individual components. Our work bridges mechanistic interpretability and model compression, using causal circuit attribution (RelP) specifically as a signal for *non-uniform sparsity allocation* in unstructured weight pruning.

3 Method

Our method consists of three stages: (1) circuit discovery to score model components, (2) sparsity allocation to convert scores into per-layer budgets, and (3) local pruning within each layer. We describe each in turn.

3.1 Stage 1: Circuit Attribution via RelP

Problem setup. Let $\mathcal{M}$ be a transformer language model with L layers, each containing H attention heads and one MLP block. We aim to assign an importance score $s_\ell \in \mathbb{R}_{\geq 0}$ to each layer $\ell \in \{0, \dots, L-1\}$ that reflects its functional contribution to the model’s language modeling capability. We explored both EAP-IG and RelP for this purpose; RelP produces more reliable scores (Section 6) and is used in all final results.

Relevance Patching (RelP). RelP (Jafari et al., 2025a) improves upon standard attribution patching by replacing the local gradient term with a propagation coefficient ρ derived from Layer-wise Relevance Propagation (LRP) (Bach et al., 2015). While attribution patching approximates the effect of patching a component n as:

$$\hat{c}^{\text{AtP}}(n) = (a_n^{\text{corrupt}} - a_n^{\text{clean}})^\top \frac{\partial \mathcal{L}}{\partial a_n} \Big|_{a_n^{\text{clean}}} \quad (1)$$

RelP replaces the gradient $\frac{\partial \mathcal{L}}{\partial a_n}$ with the LRP propagation coefficient ρ , yielding:

$$\hat{c}^{\text{RelP}}(n) = (a_n^{\text{corrupt}} - a_n^{\text{clean}})^\top \rho(\mathcal{L}(\mathcal{M}(x))) \Big|_{a_n^{\text{clean}}} \quad (2)$$

where the score is computed for a single clean–corrupted pair; in practice we average over all calibration pairs: $\hat{c}^{\text{RelP}}(n) = \frac{1}{|\mathcal{X}|} \sum_{x \in \mathcal{X}} (a_n^{\text{corrupt}} - a_n^{\text{clean}})^\top \rho(\mathcal{L}(\mathcal{M}(x))) \Big|_{a_n^{\text{clean}}}$. The propagation coefficient ρ is computed via LRP’s backward pass using transformer-specific rules: the LN-rule for Layer-Norm/RMSNorm, the Identity-rule for nonlinear activations (GELU/SiLU), and the Half-rule for multiplicative gating operations (Ali et al., 2022; Jafari et al., 2025b). These rules ensure relevance conservation and improved signal-to-noise ratio compared to raw gradients, which is particularly important for deep models where standard gradients become unreliable (Balduzzi et al., 2018). Crucially, RelP maintains the same computational cost as attribution patching—two forward passes and one backward pass—while producing more faithful attributions.

From component scores to layer scores. Given per-component scores from RelP, we aggregate to the layer level:

$$s_\ell = \sum_{h \in \{\text{attn}, \text{mlp}\}} |s_{\ell, h}| \quad (3)$$

In our experiments, we find that RelP produces more reliable layer-level scores than EAP-IG for the purpose of sparsity allocation (Section 6), and we use RelP-derived scores in all reported results unless otherwise noted.

Constructing corrupted inputs for general language modeling. RelP requires a clean–corrupted input pairing. A central challenge in applying circuit discovery to open-domain pruning is the absence of a natural “corrupted” distribution. In task-specific settings like the Indirect Object Identification (IOI) circuit (Wang et al., 2022), corrupted inputs are constructed by swapping specific named entities. For general language modeling, no such structured swap exists. We propose a two-pronged corruption strategy where the data is derived from the pile10k (Gao et al., 2020):

1. **Token-level shuffling:** For each input sequence, we randomly permute all non-BOS to-

kens, destroying positional and syntactic structure while preserving the token-level marginal distribution.

2. **Embedding noise injection:** We add Gaussian noise $\epsilon \sim \mathcal{N}(0, \sigma^2 I)$ with $\sigma = 0.05$ to the embedding layer activations during the corrupted forward pass, disrupting continuous representations.

This corruption is applied at every second call to the tokenizer within the attribution pipeline, ensuring the clean–corrupted pairing is consistent. The metric $\mathcal{L}$ is the cross-entropy loss on the clean input’s next-token predictions, so attribution scores reflect each component’s contribution to general language modeling capability rather than to any narrow task.

3.2 Stage 2: Non-Uniform Sparsity Allocation

Given per-layer importance scores $\{s_\ell\}_{\ell=0}^{L-1}$ and a target global sparsity $\bar{p}$, we need to assign per-layer sparsity ratios $\{p_\ell\}$ such that the parameter-weighted average equals $\bar{p}$:

$$\sum_{\ell=0}^{L-1} p_\ell \cdot \frac{|\theta_\ell|}{|\theta|} = \bar{p} \quad (4)$$

where $|\theta_\ell|$ is the number of prunable parameters in layer ℓ and $|\theta| = \sum_\ell |\theta_\ell|$.

We convert importance scores to sparsity ratios via an inverse softmax-temperature scheme. Layers with higher importance receive lower sparsity:

$$w_\ell = \text{softmax}\left(-\frac{s_\ell}{T}\right) = \frac{\exp(-s_\ell/T)}{\sum_j \exp(-s_j/T)} \quad (5)$$

where $T > 0$ is a temperature hyperparameter. The raw weights w_ℓ are then rescaled to satisfy the global constraint in Eq. 4:

$$p_\ell = \bar{p} \cdot \frac{w_\ell \cdot |\theta|}{\sum_j w_j \cdot |\theta_j|} \quad (6)$$

with clamping to $[0, p_{\max}]$ and iterative redistribution to ensure feasibility.

The temperature T controls the degree of non-uniformity: $T \rightarrow 0$ concentrates all pruning on the least important layers, while $T \rightarrow \infty$ recovers uniform allocation. We find $T = 5.0$ works well across sparsity levels for LLaMA-3.2-3B (Section 5.4).

3.3 Stage 3: Local Pruning via Wanda

Within each layer ℓ , we apply Wanda (Sun et al., 2024) at sparsity ratio p_ℓ . For each linear layer with weight matrix W and calibration input activations X , the importance of weight W_{ij} is:

$$I_{ij} = |W_{ij}| \cdot \|X_j\|_2 \quad (7)$$

and the bottom- p_ℓ fraction of weights by this score are set to zero. This criterion is computed from a calibration set of 128 sequences from the Pile-10K dataset (Gao et al., 2020), requiring only a single forward pass per layer.

4 Experimental Setup

Model. We evaluate on LLaMA-3.2-3B (Grattafiori et al., 2024), a 28-layer, 24-head transformer with $d_{\text{model}} = 3072$ and grouped-query attention (GQA) with 8 key-value heads.

Attribution. RelP uses the modified TransformerLens implementation of Jafari et al. (2025a) with LN-rule, Identity-rule, and Half-rule LRP propagation rules. The final sparsity allocation uses RelP-derived scores, which we find produce more stable layer rankings.

Evaluation. Perplexity is measured on the full WikiText-2 test set (Merity et al., 2016) with sequence length 2048, consistent with the Wanda evaluation protocol. We report all 141 non-overlapping segments. We additionally evaluate zero-shot accuracy on seven benchmarks: BoolQ, PIQA, HellaSwag, WinoGrande, ARC-Easy, ARC-Challenge, and OBQA.

Baselines. We compare against:

- **Wanda (uniform):** Standard Wanda with the same sparsity applied to every layer, using 128 calibration samples from Pile-10K.
- **U-shaped heuristic:** A non-uniform allocation that protects early and late layers (symmetric parabola: layers 0 and $L-1$ get score 1.0, the middle layer gets ~ 0), without any circuit knowledge.
- **Shuffled scores:** The circuit-derived layer scores randomly permuted across layers, run 5 times and averaged, to test whether the specific ranking matters.

Table 1: Perplexity ($\downarrow$) on WikiText-2 for LLaMA-3.2-3B at various sparsity levels. Circuit-Guided uses layer-only allocation with $T=5.0$, unprotected. The $\Delta\%$ column shows relative improvement of Circuit-Guided over Uniform Wanda. Best result per row in **bold**.

Sparsity	Wanda	Ours	$\Delta\%$
0%	7.81	7.81	—
10%	7.85	7.85	0.0%
20%	7.99	8.02	+0.4%
30%	8.61	8.58	−0.3%
50%	19.66	16.78	−14.7%
70%	1769.44	1072.98	−39.4%

5 Results

5.1 Main Results: Circuit-Guided vs. Uniform Pruning

Table 1 presents the core comparison. At low sparsity (10–30%), circuit-guided pruning performs comparably to uniform Wanda, which is expected: when little is being removed, the allocation strategy matters less than the local weight selection. The crossover occurs at approximately 35% sparsity. Beyond this point, circuit-guided pruning increasingly outperforms uniform allocation, with the gap widening as sparsity increases. At 50% sparsity, the improvement is 14.7% in perplexity; at 70%, 39.4%.

The practical significance is clearest in the 40–60% sparsity regime, which is the range most relevant for deployment: enough compression to matter, but not so much that the model is unusable. In this regime, circuit-guided pruning consistently preserves more language modeling capability per pruned weight.

5.2 Zero-Shot Downstream Results

Table 2 reports zero-shot accuracy across seven benchmarks at 50% and 70% sparsity. Circuit-guided pruning improves average accuracy by 1.1 percentage points at 50% sparsity and 2.6 points at 70%. The largest gains appear on BoolQ (+7.2 at 50%, +13.2 at 70%), consistent with circuit scores protecting boolean reasoning circuits. The average advantage grows with sparsity, mirroring the perplexity results.

5.3 Circuit Scores Fix Cross-Layer Comparability

Table 3 compares three allocation strategies: local Wanda (uniform per-layer sparsity), global Wanda (a single threshold across all layers),

and circuit-guided allocation. Global Wanda is catastrophic—perplexity exceeds 10,000 even at 30% sparsity because Wanda scores are not comparable across layers (early-layer magnitudes differ from late-layer). Circuit-guided pruning solves this: the normalized importance scores provide a principled cross-layer allocation, achieving performance comparable to the local baseline while enabling non-uniform distribution. The 28 percentage point accuracy gap between global Wanda (35.2%) and circuit-guided (63.1%) at 30% confirms the normalization works.

Table 3: Global Wanda vs. Local Wanda vs. Circuit-Guided (WikiText-2 PPL / Avg Zero-shot Acc).

Sp.	Local	Global	Circuit
30%	8.61 / 63.4	10140 / 35.2	8.62 / 63.1
40%	10.64 / 59.7	15149 / 34.9	10.42 / 59.7
50%	19.66 / 52.7	15093 / 35.0	17.95 / 52.9

Table 3 compares three allocation strategies: local Wanda (uniform per-layer sparsity), global Wanda (a single threshold across all layers), and circuit-guided allocation. Global Wanda is catastrophic—perplexity exceeds 10,000 even at 30% sparsity because Wanda scores are not comparable across layers (early-layer magnitudes differ from late-layer). Circuit-guided pruning solves this: the normalized importance scores provide a principled cross-layer allocation, achieving performance comparable to the local baseline while enabling non-uniform distribution. The 28 percentage point accuracy gap between global Wanda (35.2%) and circuit-guided (63.1%) at 30% confirms the normalization works.

5.4 Ablation Studies

We conduct five ablation studies to validate that the improvements are driven by the circuit-derived layer ranking, not by incidental properties of non-uniform allocation.

Ablation 1: Circuit-guided vs. U-shaped heuristic. A natural baseline for non-uniform pruning is the common intuition that early and late layers are more important (a U-shaped pattern). Table 4 shows that circuit-guided allocation consistently outperforms this heuristic, with the gap growing at higher sparsity. This indicates that the circuit scores capture layer importance patterns that differ from, and are more accurate than, simple positional heuristics.

Table 2: Zero-shot accuracy ($\uparrow$) on seven benchmarks at 50% and 70% unstructured sparsity (LLaMA-3.2-3B). Δ shows improvement of Circuit-Guided over Wanda.

Sparsity	Method	BoolQ	PIQA	HSwag	WGr	ARC-e	ARC-c	OBQA	Avg
0%	Dense	73.3	77.5	73.6	69.8	71.6	46.1	43.0	65.0
50%	Wanda	59.8	71.3	56.6	60.3	53.7	33.5	33.6	52.7
	Ours	67.0	70.3	56.2	59.9	53.5	31.8	32.0	53.8
	Δ	+7.2	-1.0	-0.4	-0.4	-0.2	-1.7	-1.6	+1.1
70%	Wanda	43.7	52.4	27.4	49.9	29.3	22.7	26.8	36.0
	Ours	56.9	53.1	27.9	50.3	29.0	23.4	29.8	38.6
	Δ	+13.2	+0.7	+0.5	+0.4	-0.3	+0.7	+3.0	+2.6

Table 4: Perplexity ($\downarrow$): Uniform vs. U-shaped vs. Circuit-Guided.¹

Sparsity	Uniform	U-Shaped	Ours
30%	8.61	8.68	8.58
50%	19.66	22.51	16.78
70%	1769.44	2682.27	1072.98

Ablation 2: Do scores work without layer protection? Our default configuration protects layers 0 and $L-1$ with a 10% sparsity cap. To test whether the circuit scores are genuinely informative or whether this heuristic protection is doing the work, we remove the cap entirely. Table 5 shows that unprotected circuit-guided pruning actually *improves* over the protected variant at all sparsity levels, achieving 16.78 and 1072.98 at 50% and 70% respectively. This confirms that the circuit scores alone carry sufficient signal to identify critical layers—manual protection is unnecessary and slightly suboptimal.

Table 5: Effect of removing layer-0/layer-27 protection. “Unprotected” uses circuit scores alone.

Sparsity	Uniform	Protected	Unprotected
30%	8.61	8.62	8.58
50%	19.66	17.95	16.78
70%	1769.44	1720.62	1072.98

Ablation 3: Shuffled scores control. To test whether the *specific* layer ranking matters or whether *any* non-uniform allocation would suffice, we randomly permute the circuit scores across layers and evaluate 5 random seeds per sparsity level. Table 6 shows that the true circuit ranking significantly outperforms random shuffles, particularly at 50% sparsity where the gap is 6.27 PPL points ($p < 0.01$). This confirms that the signal is in the specific layer ordering, not merely in the act of making allocation non-uniform.

Table 6: Circuit-guided vs. shuffled layer scores (mean $\pm$ std over 5 random permutations).¹

Sparsity	Uniform	Shuffled	Ours
30%	8.61	8.70 $\pm$ 0.03	8.58
50%	19.66	24.16 $\pm$ 1.51	16.78
70%	1769	3301 $\pm$ 2351	1073

Ablation 4: Temperature sensitivity. Table 7 reports perplexity at 30% sparsity across temperatures. Low temperatures ($T \leq 3$) produce extreme allocation and degrade performance; high temperatures ($T \geq 3$) approach uniform allocation. The optimum at $T=5.0$ represents a mildly non-uniform regime, suggesting that circuit scores are best used as a gentle prior rather than a hard assignment.

Table 7: Effect of temperature T at 30% target sparsity using the protected configuration. Uniform Wanda achieves 8.61; the unprotected optimum at $T=5.0$ is 8.58 (Table 5).

T	0.1	0.3	0.5	1.0	2.0	3.0	5.0
PPL	237	11.5	9.41	8.87	8.71	8.66	8.63

Ablation 5: Per-matrix vs. layer-only allocation. We compare three granularities of circuit-guided allocation: (a) per-matrix raw, where each of the 7 linear layers per transformer block (Q, K, V, O, gate, up, down) receives an individual sparsity ratio based on raw RelP scores, (b) per-matrix normalized, where component-type scores are scaled to mean=1.0 before allocation, and (c) layer-only, where all matrices within a layer share the same ratio. Layer-only allocation consistently outperforms both per-matrix variants (Table 8). Raw per-matrix mode is worse than uniform at

¹The U-shaped and shuffled baselines were evaluated using the protected configuration (layers 0 and 27 capped at 10% sparsity). The “Ours” column uses the unprotected configuration. This makes the comparison slightly conservative for our method; re-running these baselines unprotected would likely widen the gap, as removing protection generally increases variance for non-circuit-guided allocations.

50% because RelP’s component-type scores reflect architectural role (o_proj always scores high, k_proj scores low) rather than pruning sensitivity, over-protecting some projections while over-pruning others. Component-type normalization recovers most of the gap—perplexity drops from 22.03 to 17.96—confirming the diagnosis. But layer-only remains best because it makes only 28 allocation decisions (which layers matter) and lets Wanda handle the 196 within-layer decisions where its activation-aware scoring excels

Table 8: Per-matrix vs. layer-only circuit-guided allocation ($T=5.0$).

Sparsity	Uniform	Per-M Raw	Per-M Norm	Layer
30%	8.61	8.66	8.59	8.58
50%	19.66	22.03	17.96	16.78
70%	1769	1340	1563	1073

5.5 Recovery via Fine-tuning

Table 9 examines whether circuit-guided pruning’s advantage persists after LoRA fine-tuning (rank-8, 500 steps) at 50% sparsity. Circuit-guided pruning provides a better starting point (PPL 17.95 vs. 19.66), but uniform Wanda recovers more from LoRA fine-tuning. We hypothesize that the likely cause is: circuit-guided pruning’s heterogeneous sparsity (2.6% at layer 27, 59% at layer 16) creates uneven damage that rank-8 LoRA cannot efficiently address—heavily pruned middle layers need more capacity than the adapter provides. This parallels Wanda++’s finding that their method is orthogonal to LoRA fine-tuning (Yang et al., 2025). The practical implication is that circuit-guided pruning is most valuable in the no-recovery setting, which is the more common deployment scenario since the goal of pruning is to avoid expensive training.

Table 9: LoRA recovery at 50% sparsity (500 steps, rank-8, LLaMA-3.2-3B).

	PPL (↓)		Acc (↑)	
	Unif.	Circ.	Unif.	Circ.
Before LoRA	19.66	17.95	52.7	52.9
After LoRA	11.29	11.81	60.1	58.8

6 Discussion

What do the circuit scores reveal about layer importance? The RelP-derived layer scores for LLaMA-3.2-3B reveal a non-trivial importance landscape. Layer 27 (the final layer) receives the highest score (120.93), consistent with its role in

projecting to the output vocabulary. Layer 1 receives the second-highest score (114.34), suggesting an early processing bottleneck. The minimum occurs at mid-depth layers (e.g., layer 13 scores 78.59), which are allocated the highest sparsity. Notably, this pattern is *not* a simple U-shape: layer 0 (110.02) is less important than layer 1, and the mid-layer minimum is asymmetric, shifted toward the second quarter of the network. This explains why the U-shaped heuristic underperforms.

Where does non-uniform allocation help most?

The crossover at $\sim 35\%$ sparsity can be understood intuitively: at low sparsity, every layer has enough capacity to absorb its quota of pruning regardless of allocation. At high sparsity, uniform allocation forces critical layers below their minimum viable capacity, causing cascading errors that non-uniform allocation avoids.

EAP-IG vs. RelP for sparsity allocation. We explored both EAP-IG and RelP as the attribution method for deriving layer importance scores. Both produce qualitatively similar layer rankings, but RelP’s scores exhibit a better signal-to-noise ratio, consistent with the findings of Jafari et al. (2025a) that RelP more faithfully approximates activation patching than gradient-based methods, particularly for MLP outputs and the residual stream. Since our sparsity allocation depends on the *relative* ordering and magnitude of layer scores, RelP’s improved faithfulness translates directly to better allocation quality. All main results use RelP-derived scores.

Comparison with Wanda++. Since Wanda++ (Yang et al., 2025) reports mainly on OpenLLaMA (3B/7B/70B), LLaMA-1 (7B/13B/30B/65B) and LLaMA-3.1 (8B) rather than LLaMA-3.2-3B, a direct comparison is not possible but are open for future work. However, the relative improvements are informative: Wanda++ achieves approximately 3% perplexity reduction over Wanda at 50% unstructured sparsity on 7B, while our method achieves 14.7% on 3B. At 70%, Wanda++ achieves approximately 27% reduction versus our 39.4%. Crucially, the two approaches are orthogonal—Wanda++ improves the within-layer score (adding gradient information), while ours improves the cross-layer budget. Combining circuit-guided allocation with Wanda++’s RGS criterion is a natural and promising extension.

Limitations

Scale. We evaluate on a single model (LLaMA-3.2-3B). Experiments on larger models (7B, 13B, 70B) and other architectures (Mistral, Qwen) are necessary to assess generality.

Orthogonality with improved local criteria. Our method is orthogonal to improved within-layer pruning criteria such as Wanda++ (Yang et al., 2025); combining circuit-guided allocation with Wanda++’s RGS criterion is a natural extension.

LoRA recovery. The LoRA recovery experiment suggests that non-uniform sparsity may require non-uniform adapter capacity; exploring per-layer LoRA rank allocation guided by the same circuit scores is promising future work.

Attribution cost. The EAP-IG and RelP attribution stages add computational overhead: approximately 30 minutes on a single A100 GPU for 500 calibration samples. While this is a one-time cost amortized over any number of sparsity configurations, it is non-trivial. Cheaper attribution methods (e.g., single-step gradient approximations) could reduce this.

Corruption strategy. Our token-shuffling + noise corruption is a heuristic. More principled approaches—such as masking-based corruption aligned with the model’s pretraining objective, or using multiple corruption strategies and ensembling the resulting scores—may yield better attributions.

7 Conclusion

We have presented Circuit-Guided Pruning, a method that uses mechanistic interpretability to allocate non-uniform sparsity budgets for LLM pruning. By running Relevance Patching (RelP) attribution on a small calibration set, we derive layer-level importance scores that capture functional circuit structure. These scores are converted to per-layer sparsity ratios via a softmax-temperature scheme, while Wanda handles the fine-grained weight selection within each layer.

The key insight is that circuit discovery and weight pruning operate at complementary scales: circuit discovery excels at identifying *which layers matter*, while local pruning criteria like Wanda excel at identifying *which weights within a layer matter*. Combining both yields a method that outperforms either alone. Our results on LLaMA-3.2-3B

demonstrate consistent improvements at moderate-to-high sparsity, validated by ablation studies that confirm the circuit signal is genuine, and zero-shot evaluations across seven benchmarks that demonstrate downstream task improvements.

More broadly, this work suggests that mechanistic interpretability—often viewed as a tool for understanding models—can have direct, practical applications in model compression. We hope this encourages further work at the intersection of these two fields.

References

- Ameen Ali, Thomas Schnake, Oliver Eberle, Grégoire Montavon, {Klaus Robert} Müller, and Lior Wolf. 2022. Xai for transformers: Better explanations through conservative propagation. volume 162, pages 435–451. Publisher Copyright: Copyright © 2022 by the author(s); 39th International Conference on Machine Learning, ICML 2022 ; Conference date: 17-07-2022 Through 23-07-2022.
- S. Bach, A. Binder, G. Montavon, F. Klauschen, K.-R. Müller, and W. Samek. 2015. On pixel-wise explanations for non-linear classifier decisions by layer-wise relevance propagation. *PLoS ONE*, 10(7):e0130140.
- David Balduzzi, Marcus Frean, Lennox Leary, JP Lewis, Kurt Wan-Duo Ma, and Brian McWilliams. 2018. [The shattered gradients problem: If resnets are the answer, then what is the question?](#)
- Arthur Conmy, Augustine N. Mavor-Parker, Aengus Lynch, Stefan Heimersheim, and Adrià Garriga-Alonso. 2023. [Towards automated circuit discovery for mechanistic interpretability](#). volume abs/2304.14997.
- Fahim Dalvi, Hassan Sajjad, Nadir Durrani, and Yonatan Belinkov. 2020. [Analyzing redundancy in pretrained transformer models](#). In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 4908–4926. Online. Association for Computational Linguistics.
- N. Elhage, N. Nanda, C. Olsson, T. Henighan, N. Joseph, B. Mann, A. Askell, Y. Bai, A. Chen, T. Conerly, and 1 others. 2021. A mathematical framework for transformer circuits. *Transformer Circuits Thread*.
- Elias Frantar and Dan Alistarh. 2023. [Sparsegpt: Massive language models can be accurately pruned in one-shot](#).
- Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, Shawn Presser, and Connor Leahy. 2020. [The pile: An 800gb dataset of diverse text for language modeling](#).

- Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, Amy Yang, Angela Fan, Anirudh Goyal, Anthony Hartshorn, Aobo Yang, Archi Mitra, Archie Sravankumar, Artem Korenev, Arthur Hinsvark, and 542 others. 2024. [The llama 3 herd of models](#).
- Song Han, Jeff Pool, John Tran, and William J. Dally. 2015. [Learning both weights and connections for efficient neural networks](#).
- Michael Hanna, Sandro Pezzelle, and Yonatan Belinkov. 2024. [Have faith in faithfulness: Going beyond circuit overlap when finding model mechanisms](#).
- Babak Hassibi, David G. Stork, and Gregory J. Wolff. 1993. [Optimal brain surgeon and general network pruning](#). pages 293–299 vol.1.
- Sayed Mohammad Vakilzadeh Hatefi, Maximilian Dreyer, Reduan Achitibat, Patrick Kahardipraja, Thomas Wiegand, Wojciech Samek, and Sebastian Lapuschkin. 2025. [Attribution-guided pruning for compression, circuit discovery, and targeted correction in llms](#).
- Farnoush Rezaei Jafari, Oliver Eberle, Ashkan Khakzar, and Neel Nanda. 2025a. [Relp: Faithful and efficient circuit discovery in language models via relevance patching](#).
- Farnoush Rezaei Jafari, Grégoire Montavon, Klaus-Robert Müller, and Oliver Eberle. 2025b. [Mambalrp: Explaining selective state space sequence models](#).
- Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. 2022. [Locating and editing factual associations in gpt](#). In *Neural Information Processing Systems*.
- Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2016. [Pointer sentinel mixture models](#).
- N. Nanda and J. Bloom. 2023. Attribution patching: Activation patching at industrial scale. *Blog post*.
- Catherine Olsson, Nelson Elhage, Neel Nanda, Nicholas Joseph, Nova DasSarma, Tom Henighan, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Scott Johnston, Andy Jones, Jackson Kernion, Liane Lovitt, and 7 others. 2022. [In-context learning and induction heads](#).
- Mingjie Sun, Zhuang Liu, Anna Bair, and J. Zico Kolter. 2024. [A simple and effective pruning approach for large language models](#).
- Mukund Sundararajan, Ankur Taly, and Qiqi Yan. 2017. [Axiomatic attribution for deep networks](#).
- Aaquib Syed, Can Rager, and Arthur Conmy. 2024. [Attribution patching outperforms automated circuit discovery](#). In *Proceedings of the 7th BlackboxNLP Workshop: Analyzing and Interpreting Neural Networks for NLP*, pages 407–416, Miami, Florida, US. Association for Computational Linguistics.
- Jesse Vig, Sebastian Gehrmann, Yonatan Belinkov, Sharon Qian, Daniel Nevo, Yaron Singer, and Stuart Shieber. 2020. Investigating gender bias in language models using causal mediation analysis. volume 2020-December. Neural information processing systems foundation. Publisher Copyright: © 2020 Neural information processing systems foundation. All rights reserved.; 34th Conference on Neural Information Processing Systems, NeurIPS 2020 ; Conference date: 06-12-2020 Through 12-12-2020.
- Elena Voita, David Talbot, Fedor Moiseev, Rico Senrich, and Ivan Titov. 2019. [Analyzing multi-head self-attention: Specialized heads do the heavy lifting, the rest can be pruned](#). In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 5797–5808, Florence, Italy. Association for Computational Linguistics.
- Kevin Wang, Alexandre Variengien, Arthur Conmy, Buck Shlegeris, and Jacob Steinhardt. 2022. [Interpretability in the wild: a circuit for indirect object identification in gpt-2 small](#).
- Yifan Yang, Kai Zhen, Bhavana Ganesh, Aram Galstyan, Goeric Huybrechts, Markus Müller, Jonas M. Kübler, Rupak Vignesh Swaminathan, Athanasios Mouchtaris, Sravan Babu Bodapati, Nathan Susanj, Zheng Zhang, Jack FitzGerald, and Abhishek Kumar. 2025. [Wanda++: Pruning large language models via regional gradients](#). In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 4321–4333, Vienna, Austria. Association for Computational Linguistics.
- Lu Yin, You Wu, Zhenyu Zhang, Cheng-Yu Hsieh, Yaqing Wang, Yiling Jia, Gen Li, Ajay Jaiswal, Mykola Pechenizkiy, Yi Liang, Michael Bendersky, Zhangyang Wang, and Shiwei Liu. 2025. [Outlier weighed layerwise sparsity \(owl\): A missing secret sauce for pruning llms to high sparsity](#).